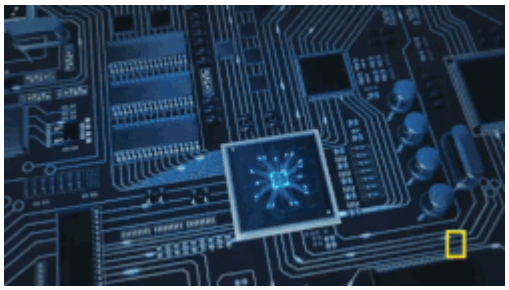
Minimierung von Boolesche Funktionen / Schaltnetze

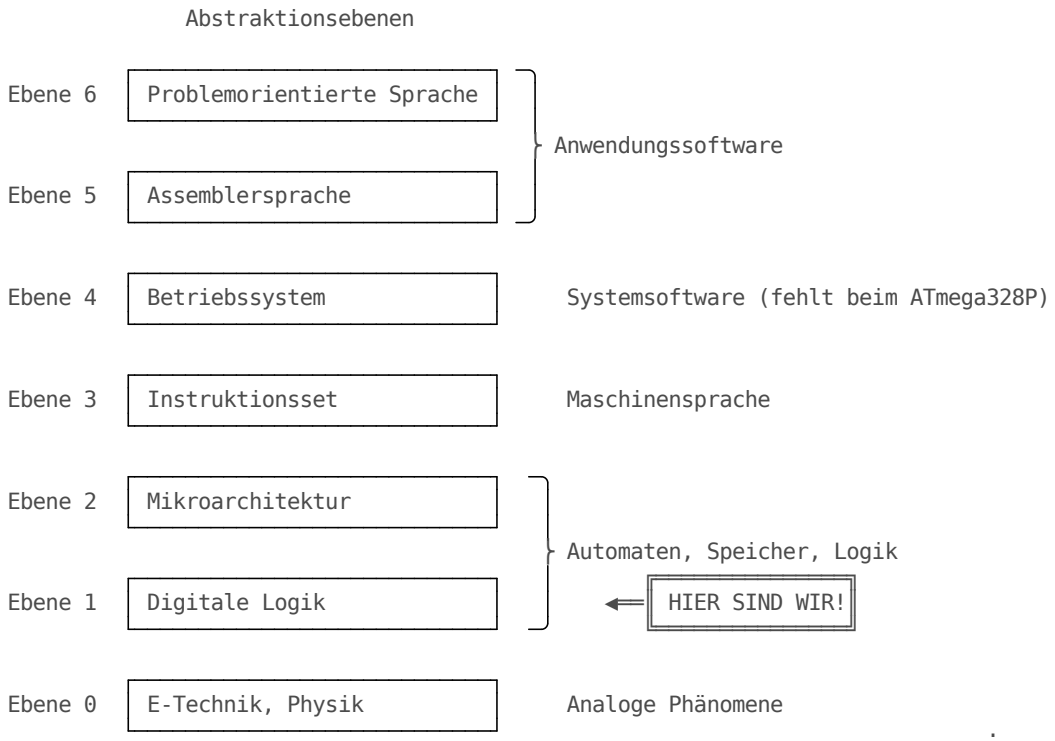
Parameter	Kursinformationen
Veranstaltung:	`Digitale Systeme / Eingebettete Systeme`
Semester:	`Wintersemester 2026/27`
Hochschule:	`Technische Universität Freiberg`
Inhalte:	`Verfahren von Quine-McCluskey, Realisierung von Schaltnetzen`
Link auf GitHub:	https://github.com/TUBAF-IfI-LiaScript/VL_EingebetteteSysteme/blob/master/04_Schaltnetze.md
Autoren:	Sebastian Zug & André Dietrich & Fabian Bär & Copilot & Claude Code (Opus 5.5)



Lernziele: Nach dieser Vorlesung können Sie

- das Verfahren nach Quine-McCluskey anwenden: Primimplikanten bestimmen und mit der Primimplikantentafel eine minimale Überdeckung auswählen,
- eine minimierte Schaltfunktion als zweistufiges Schaltnetz umsetzen – mit UND, ODER und NICHT oder nur mit NAND-Gattern,
- erklären, wann PROM und PAL zur Umsetzung von Schaltnetzen eingesetzt werden,
- Glitches als Folge unterschiedlicher Signallaufzeiten erkennen und mit einem zusätzlichen Term aus dem Karnaugh-Veitch-Diagramm vermeiden.

🗺️ Diese Vorlesung in der Kurslandkarte →



Feedbacks/Ergänzungen

Format	Informatik Studierende	Nicht-Informatik Studierende
Verbesserungsvorschlag	0	2
Fragen	2	1
generelle Hinweise	1	1

https://github.com/TUBAF-Ifi-LiaScript/VL_EingebetteteSysteme/tree/master/exampleCode/03_Minimierung



Was gefällt Ihnen daran, wo sehen Sie ggf. Verbesserungsbedarf?

Können wir eine Übersicht bekommen, die die Formeln in der hier verwendeten und traditionellen Schreibweise gegenüberstellt?

ja, kommt auf Seite 12.

Beispielanwendung

Nehmen wir an, dass wir die Logik einer Kaffeemaschine umsetzen wollen. Diese verfügt über verschiedene Eingänge wie Sensoren an der Heizplatte, einem Füllstandsmesser, Drucksensorik usw. Wir gehen davon aus, dass diese lediglich digitale Ausgaben generiert.

Die Variable yy gibt entsprechend einen Fehlerzustand wieder und ist zum Beispiel mit einer LED verknüpft. Ab Zustand 6 bis 14 soll diese Leuchten.


12
10
8
6

x_3**x3**	x_2**x2**	x_1**x1**	x_0**x0**	y**y**	**Zustand**
0	0	0	0	0	0 - Initialisierung
0	0	0	1	0	1 - Heizplatte erwärmen
0	0	1	0	0	2 - Wasserkocher an
0	0	1	1	0	3 -
0	1	0	0	0	4 -
0	1	0	1	0	5 -
0	1	1	0	1	6 - Wasser fehlt
0	1	1	1	1	7 - Kaffeefach geöffnet
1	0	0	0	1	8 - Druckabfall
1	0	0	1	1	9 - …
1	0	1	0	1	10 -
1	0	1	1	1	11 -
1	1	0	0	1	12 -

1	1	0	1	1	13 -
1	1	1	0 dlr	1	14 - Wartung fällig
1	1	1	1	0	15 - Kaffee fertig

Wie muss also die Schaltung für diese Aufgabe umgesetzt werden?

Schritt 1: Aufstellen der Gleichungen

Kanonische Disjunktive Normalform (KDNF)

- eindeutige Darstellung einer booleschen Funktion f als Disjunktion von Mintermen
- Beispiel: $(x \cdot y \cdot z) + (x \cdot y \cdot z) + (x \cdot y \cdot z)$(x · y · z) + (x · y · z) + (x · y · z) ist KDNF von $f(x, y, z)$f(x, y, z)

Kanonische Konjunktive Normalform (KKNF)

- eindeutige Darstellung einer booleschen Funktion f als Konjunktion von Maxtermen
- Beispiel: $(x + y) \cdot (x + y) \cdot (x + y)$(x + y) · (x + y) · (x + y) ist KKNF von $f(x, y)$f(x, y)

x_3x3	x_2x2	x_1x1	x_0x0	yy	Minterm	Maxterm
0	0	0	0	0	???	???
0	0	0	1	0		
0	0	1	0	0		
0	0	1	1	0		
0	1	0	0	0		
0	1	0	1	0		
0	1	1	0	1		
0	1	1	1	1		
1	0	0	0	1		
1	0	0	1	1		
1	0	1	0	1		
1	0	1	1	1		
1	1	0	0	1		
1	1	0	1	1		
1	1	1	0	1		
1	1	1	1	0		

x_3x3	x_2x2	x_1x1	x_0x0	yy
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	1
0	1	1	1	1
1	0	0	0	1
1	0	0	1	1
1	0	1	0	1

1	0	1	1	1
1	1	0	0	1
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0

Schritt 2: Vereinfachen der Minterme - Analytische Lösung

$$
\begin{aligned}
y = \quad & \bar{x_3}x_2x_1\bar{x_0} + \bar{x_3}x_2x_1x_0 + \\
& x_3\bar{x_2}\bar{x_1}\bar{x_0} + x_3\bar{x_2}\bar{x_1}x_0 + \\
& x_3\bar{x_2}x_1\bar{x_0} + x_3\bar{x_2}x_1x_0 + \\
& x_3x_2\bar{x_1}\bar{x_0} + x_3x_2\bar{x_1}x_0 + \\
& x_3x_2x_1\bar{x_0}
\end{aligned}
$$

y =x3x2x1x0 + x3x2x1x0+x3x2 x1 x0 + x3x2 x1x0+x3x2x1x0 + x3x2x1x0+x3x2x1 x0 + x3x2x1x0+x3

x2x1x0

Wie gehen Sie vor? Wir suchen Paare von Mintermen, die sich lediglich in einer Variablen unterscheiden, und fassen diese entsprechd dem Distributivgesetz und Idempotenzgesetz zusammen.

$$
\begin{aligned}
\bar{x_3}x_2x_1\bar{x_0} + \bar{x_3}x_2x_1x_0 \quad & = \bar{x_3}x_2x_1(\bar{x_0} + x_0) \\
& = \bar{x_3}x_2x_1(1) \\
& = \bar{x_3}x_2x_1
\end{aligned}
$$

x3x2x1x0 + x3x2x1x0 = x3x2x1(x0 + x0) = x3x2x1(1) = x3x2x1

Erste Stufe der Vereinfachung dlr Zunächst erweitern wir unseren KDNF um einen der Minterme, so dass die möglichen Minimierungen pro Zeile offensichtlich sind.

$$\begin{aligned} y = \quad & \bar{x}_3 x_2 x_1 \bar{x}_0 + \bar{x}_3 x_2 x_1 x_0 + \\ & x_3 \bar{x}_2 \bar{x}_1 \bar{x}_0 + x_3 \bar{x}_2 \bar{x}_1 x_0 + \\ & x_3 \bar{x}_2 x_1 \bar{x}_0 + x_3 \bar{x}_2 x_1 x_0 + \\ & x_3 x_2 \bar{x}_1 \bar{x}_0 + x_3 x_2 \bar{x}_1 x_0 + \\ & x_3 \bar{x}_2 x_1 \bar{x}_0 + x_3 x_2 x_1 \bar{x}_0 \end{aligned}$$

y =x3x2x1x0 + x3x2x1x0+x3x2 x1 x0 + x3x2 x1x0+x3x2x1x0 + x3x2x1x0+x3x2x1 x0 + x3x2x1x0+x3

x2x1x0 + x3x2x1x0

Damit ergibt sich die oben genannten Gleichung in der ersten Vereinfachungsstufe zu

$$y = \quad \bar{x}_3 x_2 x_1 + x_3 \bar{x}_2 \bar{x}_1 + x_3 \bar{x}_2 x_1 + x_3 x_2 \bar{x}_1 + x_3 x_1 \bar{x}_0$$

y =x3x2x1 + x3x2 x1 + x3x2x1 + x3x2x1 + x3x1x0

Zweite Stufe der Vereinfachung

$$\begin{aligned} y = \quad & \bar{x}_3 x_2 x_1 + \\ & x_3 \bar{x}_2 \bar{x}_1 + \\ & x_3 \bar{x}_2 x_1 + \\ & x_3 x_2 \bar{x}_1 + \\ & x_3 x_1 \bar{x}_0 \end{aligned}$$

y =x3x2x1+x3x2 x1+x3x2x1+x3x2x1+x3x1x0

$$y = \bar{x}_3 x_2 x_1 + x_3 \bar{x}_2 + x_3 \bar{x}_1 + x_3 x_1 \bar{x}_0$$

y = x3x2x1 + x3 x2 + x3x1 + x3x1x0

Gegenprobe I

Minimize.py

```
from sympy.logic import SOPform
from sympy import symbols
x3, x2, x1, x0 = symbols('x3 x2 x1 x0')

minterms = [[0, 1, 1, 0],
            [0, 1, 1, 1],
            [1, 0, 0, 0],
            [1, 0, 0, 1],
            [1, 0, 1, 0],
            [1, 0, 1, 1],
            [1, 1, 0, 0],
            [1, 1, 0, 1],
            [1, 1, 1, 0]]
result = SOPform([x3, x2, x1, x0], minterms)
print(result)
```

```
(x3 & ~x0) | (x3 & ~x1) | (x3 & ~x2) | (x1 & x2 & ~x3)
```

Überrascht? Offenbar gelingt es dem Minimierungsansatz von *sympy*, eine kompaktere Form zu finden.

Schauen wir uns die Funktion im Karnaugh-Diagramm an!

	$\bar{x}_1\bar{x}_0$ **x1** **x0**	$\bar{x}_1 x_0$ **x1** **x0**	$x_1 x_0$ **x1** **x0**	$x_1\bar{x}_0$ **x1** **x0**
$\bar{x}_3\bar{x}_2$ x3 x2	0	0	0	0
$\bar{x}_3 x_2$ x3 x2	0	0	1	1
$x_3 x_2$ x3 x2	1	1	0	1
$x_3\bar{x}_2$ x3 x2	1	1	1	1

$$y_{Karnaugh} = \bar{x}_3 x_2 x_1 + x_3 \bar{x}_2 + x_3 \bar{x}_1 + x_3 \bar{x}_0$$
$$y_{Analytic} = \bar{x}_3 x_2 x_1 + x_3 \bar{x}_2 + x_3 \bar{x}_1 + x_3 x_1 \bar{x}_0$$

yKarnaughyAnalytic = x3x2x1 + x3 x2 + x3x1 + x3x0 = x3x2x1 + x3 x2 + x3x1 + x3x1x0

Warum „übersieht“ unsere analytische Lösung die mögliche Minimierung im letzten Term?

> Weil wir uns in der Vereinfachung der ersten Stufe mit einem Teilergebnis zufriedengegeben haben, das uns in der zweiten Stufe für eine weiteren Aggregation fehlt.

Welche Kombinationen sind zusätzlich möglich?

$$\begin{aligned} y = \quad & \bar{x}_3 x_2 x_1 \bar{x}_0 + \bar{x}_3 x_2 x_1 x_0 + \\ & x_3 \bar{x}_2 \bar{x}_1 \bar{x}_0 + x_3 \bar{x}_2 \bar{x}_1 x_0 + \\ & x_3 \bar{x}_2 x_1 \bar{x}_0 + x_3 \bar{x}_2 x_1 x_0 + \\ & x_3 x_2 \bar{x}_1 \bar{x}_0 + x_3 x_2 \bar{x}_1 x_0 + \\ & x_3 \bar{x}_2 x_1 \bar{x}_0 + x_3 x_2 x_1 \bar{x}_0 \end{aligned}$$

y =x3x2x1x0 + x3x2x1x0+x3x2 x1 x0 + x3x2 x1x0+x3x2x1x0 + x3x2x1x0+x3x2x1 x0 + x3x2x1x0+x3

x2x1x0 + x3x2x1x0

$$y = \quad \bar{x}_3 x_2 x_1 + x_3 \bar{x}_2 \bar{x}_1 + x_3 \bar{x}_2 x_1 + x_3 x_2 \bar{x}_1 + x_3 x_1 \bar{x}_0 + x_3 \bar{x}_1 \bar{x}_0$$

y =x3x2x1 + x3x2 x1 + x3x2x1 + x3x2x1 + x3x1x0 + x3x1 x0

Daraus ergibt sich dann auf der zweiten Stufe die „zusätzliche“ Minimierungsoption, die die beiden letzten Minterme zusammenfasst.

$$y = \quad \bar{x}_3 x_2 x_1 + x_3 \bar{x}_2 \bar{x}_1 + x_3 \bar{x}_2 x_1 + x_3 x_2 \bar{x}_1 + \underbrace{x_3 x_1 \bar{x}_0 + x_3 \bar{x}_1 \bar{x}_0}$$
$$y = \quad \bar{x}_3 x_2 x_1 + x_3 \bar{x}_2 + x_3 \bar{x}_1 + x_3 \bar{x}_0$$

y =y =x3x2x1 + x3x2 x1 + x3x2x1 + x3x2x1 +

---

x3x1x0 + x3x1 x0x3x2x1 + x3 x2 + x3x1 + x3x0

Erkenntnisse:

- Das Karnaugh-Veitch-Diagramm zeigt mögliche Minimierungspotentiale auf, hinsichtlich der Bildung der Schleifen können unterschiedliche Strategien zum Tragen kommen.
- Offenkundig brauchen wir ein systematischeres Vorgehen bei der Vereinfachung, das alle Kombinationen möglicher Terme berücksichtigt.

Verfahren nach Quine-McCluskey

Das Verfahren bezieht sich zunächst nur auf Funktionsdarstellungen in kanonischer disjunktiver Normalform (KDNF) und zielt darauf ab eine systematische Minimierung der Minterme durchzuführen.

Ermittlung der Primterme

Die Minterme werden in tabellarischer Form entsprechend der Zahl der „1“en sortiert. Es gibt 1 Eintrag mit einer 1 (m_8m8) und jeweils 4 mit 2 oder 3 Einsen.

	x_3 **x 3**	x_2 **x 2**	x_1 **x 1**	x_0 **x 0**	y **y**
	0	0	0	0	0
	0	0	0	1	0
	0	0	1	0	0
	0	0	1	1	0
	0	1	0	0	0
	0	1	0	1	0
m_6 m 6	0	1	1	0	1
m_7 m 7	0	1	1	1	1
m_8 m 8	1	0	0	0	1
m_9 m 9	1	0	0	1	1
m_{10} m 1 0	1	0	1	0	1

m_x **m x**	x_3 **x 3**	x_2 **x 2**	x_1 **x 1**	x_0 **x 0**	**O K**
m_8 m8	1 1	0 0	0 0	0 0	.
m_6 m 6	0 0	1	1	0	.
m_9 m 9	1	0	0	1	.
m_{10} m 1 0	1	0	1	0	.
m_{12} m 1 2	1	1	0	0	.
m_7 m7	0 0	1 1	1 1	1 1	.
m_{11} m11	1 1	0 0	1 1	1 1	.
m_{13} m13	1 1	1 1	0 0	1 1	.
m_{14} m14	1 1	1 1	1 1	0 0	.

m_{11} m 1 1	1	0	1	1	1
m_{12} m 1 2	1	1	0	0	1
m_{13} m 1 3	1	1	0	1	1
m_{14} m 1 4	1	1	1	0	1
	1	1	1	1	0

In einem zweiten Schritt werden die sortierten Minterme evaluiert. Dabei können nur die Blöcke blau-weiß und weiß-grau potentiell zusammenfassbare Mintermpaare umfassen.

	x_3**x3**	x_2**x2**	x_1**x1**	x_0**x0**	**OK**
$m_8 \cdot m_9$ m8 · m9	1	0	0	-	.
$m_8 \cdot m_{10}$ m8 · m10	1	0	-	0	.
$m_8 \cdot m_{12}$ m8 · m12	1	-	0	0	.
$m_6 \cdot m_7$ m6 · m7	00	11	11	– –	.
$m_6 \cdot m_{14}$ m6 · m14	– –	11	11	00	.
$m_9 \cdot m_{11}$ m9 · m11	11	00	– –	11	.
$m_9 \cdot m_{13}$ m9 · m13	11	– –	00	11	.
$m_{10} \cdot m_{11}$ m10 · m11	11	00	11	– –	.
$m_{10} \cdot m_{14}$ m10 · m14	11	– –	11	00	.
$m_{12} \cdot m_{13}$ m12 · m13	11	11	00	– –	.
$m_{12} \cdot m_{14}$ m12 · m14	11	11	– –	00	.

Kombinations.py

```
1  import numpy as np
2  minterms = [[0, 1, 1, 0],
3              [0, 1, 1, 1],
4              [1, 0, 0, 0],
5              [1, 0, 0, 1],
6              [1, 0, 1, 0],
7              [1, 0, 1, 1],
8              [1, 1, 0, 0],
9              [1, 1, 0, 1],
10             [1, 1, 1, 0]]
11 distances = np.zeros([len(minterms),len(minterms)]).astype('float')
12 for i in range(0, len(minterms)):
13     for k in range(0, i):
14         diff = np.subtract(minterms[i], minterms[k])
15         dist = np.sum(np.abs(diff))
16         distances[i,k] = dist
17
18 print("Distanzen der Minterme")
19 for j in range(0, len(distances)):  # Requiered instead print(distanc
20     print(distances[j])             # due to pyscript output constrai
21 print("Kombinationen mit Distanz 1: {}".format(np.count_nonzero(dista
     == 1)))
```

```
Distanzen der Minterme
[0. 0. 0. 0. 0. 0. 0. 0. 0.]
[1. 0. 0. 0. 0. 0. 0. 0. 0.]
[3. 4. 0. 0. 0. 0. 0. 0. 0.]
[4. 3. 1. 0. 0. 0. 0. 0. 0.]
[2. 3. 1. 2. 0. 0. 0. 0. 0.]
[3. 2. 2. 1. 1. 0. 0. 0. 0.]
[2. 3. 1. 2. 2. 3. 0. 0. 0.]
[3. 2. 2. 1. 3. 2. 1. 0. 0.]
[1. 2. 2. 3. 1. 2. 1. 2. 0.]
Kombinationen mit Distanz 1: 11
```

Die zweite Stufe wiederholt die Vorgänge - Sortierung und Evaluation erneut.

	x_3**x3**	x_2**x2**	x_1**x1**	x_0**x0**
$m_8 \cdot m_9$m8 · m9	1	0	0	-
$m_8 \cdot m_{10}$m8 · m10	1	0	-	0
$m_8 \cdot m_{12}$m8 · m12	1	-	0	0
$m_6 \cdot m_7$m6 · m7	00	11	11	– –
$m_6 \cdot m_{14}$m6 · m14	– –	11	11	00
$m_9 \cdot m_{11}$m9 · m11	11	00	– –	11
$m_9 \cdot m_{13}$m9 · m13	11	– –	00	11
$m_{10} \cdot m_{11}$ m10 · m11	11	00	11	– –
$m_{10} \cdot m_{14}$ m10 · m14	11	– –	11	00
$m_{12} \cdot m_{13}$ m12 · m13	11	11	00	– –
$m_{12} \cdot m_{14}$ m12 · m14	11	11	– –	00

Es dürfen nur Zeilen zusammengefasst werden, die die „-“ an den gleichen Positionen haben (!). Außerdem tauchen ab der zweiten Zusammenfassung (also in der dritten Tabelle) Konjunktionen doppelt auf, welche aber nur einmal notiert werden.

	x_3**x3**	x_2**x2**	x_1**x1**	x_0**x0**
$m_8 \cdot m_9 \cdot m_{10} \cdot m_{11}$ m8 · m9 · m10 · m11	1	0	-	-
$m_8 \cdot m_9 \cdot m_{12} \cdot m_{13}$ m8 · m9 · m12 · m13	1	-	0	-
$m_8 \cdot m_{10} \cdot m_9 \cdot m_{11}$ m8 · m10 · m9 · m11	1	0	-	-
$m_8 \cdot m_{10} \cdot m_{12} \cdot m_{14}$ m8 · m10 · m12 · m14	1	-	-	0
$m_8 \cdot m_{12} \cdot m_9 \cdot m_{13}$ m8 · m12 · m9 · m13	1	-	0	-
$m_8 \cdot m_{12} \cdot m_{10} \cdot m_{14}$ m8 · m12 · m10 · m14	1	-	-	0

Eine weitere Minimierung ist offenbar nicht möglich. Offenbar können 3 Minterme der ersten Stufe nicht weiter zusammengefasst werden.

	x_3x3	x_2x2	x_1x1	x_0x0
$m_8 \cdot m_9 \cdot m_{10} \cdot m_{11}$ m8 · m9 · m10 · m11	1	0	-	-
$m_8 \cdot m_9 \cdot m_{12} \cdot m_{13}$ m8 · m9 · m12 · m13	1	-	0	-
$m_8 \cdot m_{10} \cdot m_9 \cdot m_{11}$ m8 · m10 · m9 · m11	1	0	-	-
$m_8 \cdot m_{10} \cdot m_{12} \cdot m_{14}$ m8 · m10 · m12 · m14	1	-	-	0
$m_8 \cdot m_{12} \cdot m_9 \cdot m_{13}$ m8 · m12 · m9 · m13	1	-	0	-
$m_8 \cdot m_{12} \cdot m_{10} \cdot m_{14}$ m8 · m12 · m10 · m14	1	-	-	0

Mit den übrigen Ergebnissen der zweiten Stufe ergeben sich daraus insgesamt 5 Primimplikanten.

$$
\begin{aligned}
P_1 &= \quad m_8 \bullet m_9 \bullet m_{10} \bullet m_{11} \\
P_2 &= \quad m_8 \bullet m_9 \bullet m_{12} \bullet m_{13} \\
P_3 &= \quad m_8 \bullet m_{10} \bullet m_{12} \bullet m_{14} \\
P_4 &= \quad m_6 \bullet m_7 \\
P_5 &= \quad m_6 \bullet m_{14}
\end{aligned}
$$

P1 =P2 =P3 =P4 =P5 =m8 · m9 · m10 · m11m8 · m9 · m12 · m13m8 · m10 · m12 · m14m6 · m7m6

· m14

Darstellung der Minterme im Karnaugh-Diagramm

$$
\begin{aligned}
P_1 = &\quad m_8 \bullet m_9 \bullet m_{10} \bullet m_{11} \\
P_2 = &\quad m_8 \bullet m_9 \bullet m_{12} \bullet m_{13} \\
P_3 = &\quad m_8 \bullet m_{10} \bullet m_{12} \bullet m_{14} \\
P_4 = &\quad m_6 \bullet m_7 \\
P_5 = &\quad m_6 \bullet m_{14}
\end{aligned}
$$

P1 =P2 =P3 =P4 =P5 =m8 · m9 · m10 · m11m8 · m9 · m12 · m13m8 · m10 · m12 · m14m6 · m7m6

· m14

	$\bar{x}_1\bar{x}_0$**x1 x0**	$\bar{x}_1x_0$**x1x0**	x_1x_0**x1x0**	$x_1\bar{x}_0$**x1x0**
$\bar{x}_3\bar{x}_2$x3 x2	0	0	0	0
$\bar{x}_3x_2$x3x2	0	0	1 (m_7m7)	1 (m_6m6)
x_3x_2x3x2	1 (m_{12}m12)	1 (m_{13}m13)	0	1 (m_{14}m14)
$x_3\bar{x}_2$x3x2	1 (m_8m8)	1 (m_9m9)	1 (m_{11}m11)	1 (m_{10}m10)

Visualisierung der generierten Primimplikanten

	$\bar{x}_1\bar{x}_0$**x1 x0**	$\bar{x}_1x_0$**x1x0**	x_1x_0**x1x0**	$x_1\bar{x}_0$**x1x0**
$\bar{x}_3\bar{x}_2$x3 x2	0	0	0	0
$\bar{x}_3x_2$x3x2	0	0	1 (P_4P4)	1 (P_4P4, P_5 P5)
x_3x_2x3x2	1 (P_2P2, P_3 P3)	1 (P_2P2)	0	1 (P_3P3, P_5 P5)
$x_3\bar{x}_2$x3x2	1 (P_1P1, P_2 P2, P_3P3)	1 (P_1P1, P_2 P2)	1 (P_1P1)	1 (P_1P1, P_3 P3)

Offenbar werden die Minterme bis auf zwei Ausnahmen mehrfach durch die Primimplikaten abgedeckt. Hier ist eine weitere Minimierung notwendig.

Die als zweite Quine'sche Tabelle bezeichnete Primimplikatentafel fasst die Primimplikanten und die zugehörigen Minterme zusammen.

	m_{14} **m 14**	m_{13} **m 13**	m_{12} **m 12**	m_{11} **m 11**	m_{10} **m 10**	m_9 **m 9**	m_8 **m 8**	m_7 **m 7**	m_6 **m 6**
P_1 P 1				x	x	x	x		
P_2 P 2		x	x			x	x		
P_3 P 3	x		x		x		x		
P_4 P 4								x	x
P_5 P 5	x								x

Lösung der minimalen Überdeckung - intuitiv

Die Minterme m_{13}m13, m_{11}m11 und m_7m7 kommen nun in jeweils einem Primimplikanten vor. Folglich sind diese sogenannte essentielle Primimplikanten (P_1, P_2, P_4P1, P2, P4).

	m_{14} **m 14**	m_{13} **m 13**	m_{12} **m 12**	m_{11} **m 11**	m_{10} **m 10**	m_9 **m 9**	m_8 **m 8**	m_7 **m 7**	m_6 **m 6**
P_1 P1				x	x	x	x		
P_2 P2		x	x			x	x		
P_3 P 3	x		x		x		x		
P_4 P4								x	x
P_5 P 5	x								x

	$\bar{x}_1\bar{x}_0$**x1 x0**	$\bar{x}_1x_0$**x1x0**	x_1x_0**x1x0**	$x_1\bar{x}_0$**x1x0**
$\bar{x}_3\bar{x}_2$x3 x2	0	0	0	0
$\bar{x}_3x_2$x3x2	0	0	m_7m7 (P_4P4)	m_6m6 (P_4P4, P_5P5)
x_3x_2x3x2	m_{12}m12 (P_2P2, P_3P3)	m_{13}m13 (P_2P2)	0	m_{14}m14 (P_3P3, P_5P5)
$x_3\bar{x}_2$x3x2	m_8m8 (P_1P1, P_2P2, P_3P3)	m_9m9 (P_1P1, P_2P2)	m_{11}m11 (P_1P1)	m_{10}m10 (P_1P1, P_3P3)

Statement 1: m_{13}m13 ist in P_2P2 enthalten und schließt damit m_{12}m12, m_9m9 und m_8m8 ein. Diese können gestrichen werden.

	m_{14} **m14**	m_{13} **m13**	m_{11} **m11**	m_{10} **m10**	m_{7} **m7**	m_{6} **m6**
P_1 P1			x	x		
P_2 P2		x				
P_3 P3	x			x		
P_4 P4					x	x
P_5 P5	x					x

Statement 2: m_{11}m11 ist nur in P_1P1 enthalten und schließt damit m_{10}m10 ein.

Statement 3: P_4P4 deckt automatisch m_6m6 ab.

	m_{14} **m 14**	m_{13} **m 1 3**	m_{11} **m 11**	m_{7} **m 7**
P_1 P 1			x	
P_2 P 2		x		
P_3 P 3	x			
P_4 P 4				x
P_5 P 5	x			

P_3P3 und P_5P5 decken ein und den selben Minterm ab. Welcher sollte gewählt werden?

$$
\begin{aligned}
P_1 = &\quad m_8 \bullet m_9 \bullet m_{10} \bullet m_{11} \\
P_2 = &\quad m_8 \bullet m_9 \bullet m_{12} \bullet m_{13} \\
P_3 = &\quad m_8 \bullet m_{10} \bullet m_{12} \bullet m_{14} \\
P_4 = &\quad m_6 \bullet m_7 \\
P_5 = &\quad m_6 \bullet m_{14}
\end{aligned}
$$

P1 =P2 =P3 =P4 =P5 =m8 · m9 · m10 · m11m8 · m9 · m12 · m13m8 · m10 · m12 · m14m6 · m7m6

· m14

Ergebnis: Unsere relevanten Primimplikanten sind
$y = P_1 + P_2 + P_3 + P_4 = \bar{x}_3 x_2 x_1 + x_3 \bar{x}_2 + x_3 \bar{x}_1 + x_3 \bar{x}_0$y = P1 + P2 + P3 + P4 = x3x2x1 + x3 x2 + x3x1 + x3x0.

Lösung der minimalen Überdeckung - algorithmisch

Spaltendominanzprüfung

Die Spalten werden paarweise darauf verglichen, ob nicht eine Spalte existiert, in der die markierten Primterme eine Teilmenge der markierten Primterme der anderen Spalte sind. Ist dies der Fall, so kann die Spalte mit der Obermenge gestrichen werden, denn es müssen alle Konjunktionen erfasst werden und daher ist die Konjunktion mit der Obermenge durch Auswahl der Konjunktion mit der Teilmenge ebenfalls erfasst.

Beispiel Spaltendominanz:

Bei der **Spaltendominanzprüfung** werden die Spalten der Primimplikantentafel paarweise miteinander verglichen. Jede Spalte repräsentiert dabei einen **Minterm** und enthält Markierungen („x“), die angeben, **welche Primimplikanten** diesen Minterm abdecken.

Eine Spalte ($m_i$mi) **dominiert** eine andere Spalte ($m_j$mj), wenn alle „x“ in ($m_i$mi) auch in ($m_j$mj) vorkommen, also:

$$m_i \subseteq m_j$$

mi ⊆ mj

In diesem Fall kann die **größere** (also die von ($m_i$mi) dominierte) Spalte ($m_j$mj) gestrichen werden, denn sobald ($m_i$mi) abgedeckt ist, ist ($m_j$mj) automatisch ebenfalls abgedeckt. Die dominierte Spalte liefert also keine zusätzliche Bedingung für die Auswahl der Primimplikanten.

	(m_1m1)	**(m_2m2)**	**(m_3m3)**
(P_APA)	x	x	
(P_BPB)	x		x
Dominanzprüfung	streichen		

Wir lesen die Spalten als Mengen der abdeckenden Primimplikanten:

- ($m_1 = \{P_A, P_B\}$m1 = {PA, PB})
- ($m_2 = \{P_A\}$m2 = {PA})
- ($m_3 = \{P_B\}$m3 = {PB})

Da ($m_2 \subset m_1$m2 ⊂ m1) gilt, wird **(m_1m1)** gestrichen. Jede Abdeckung, die (m_2m2) erfüllt, deckt automatisch auch (m_1m1) ab.

Zeilendominanzprüfung

Bei der **Zeilendominanzprüfung** werden die **Zeilen** der Primimplikantentafel paarweise miteinander verglichen. Jede Zeile repräsentiert dabei einen **Primimplikanten** und enthält Markierungen („x“), die angeben, **welche Minterme** dieser Primimplikant abdeckt.

Eine Zeile (P_iPi) **dominiert** eine andere Zeile (P_jPj), wenn alle „x“ in (P_jPj) auch in (P_iPi) vorkommen, also:

$$P_j \subseteq P_i$$

Pj ⊆ Pi

In diesem Fall kann die **kleinere** (also die von (P_iPi) dominierte) Zeile (P_jPj) gestrichen werden, denn (P_iPi) deckt **alle** Minterme ab, die (P_jPj) abdeckt — und eventuell noch mehr. Damit ist (P_jPj) überflüssig.

	(m_1m1)	**(m_2m2)**	**(m_3m3)**	**Dominanzprüfun**
(P_APA)	x	x		streichen
(P_BPB)	x	x	x	

Wir lesen die Zeilen als Mengen der abgedeckten Minterme:

- ($P_A = \{m_1, m_2\}$PA = {m1, m2})
- ($P_B = \{m_1, m_2, m_3\}$PB = {m1, m2, m3})

Da ($P_A \subset P_B$PA ⊂ PB) gilt, wird **(P_APA)** gestrichen. (P_BPB) deckt alle Minterme ab, die auch (P_APA) abdeckt, und zusätzlich noch (m_3m3).

> Für das Verfahren von Quine-McCluskey exisitieren webbasierte Lösungen. Evaluieren Sie den Ablauf mit https://www.goldi-labs.de/SANE/view6

Schaltungssynthese

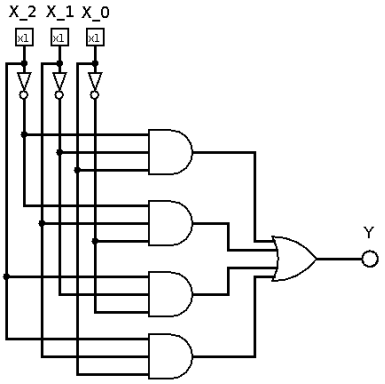
{x}*2*\, *overline{x}_1* + *x*3 \overline{x}_2 x_1 + x*3 x*2\overline{x}_1 + x*3 x*1\overline{x}_0 \end{aligned}
3 − 5 * * *

Beispiel

$y = f(x_0, x_1, x_2)$y = f(x0, x1, x2)

Die Funktion evaluiert die Parität der Variablen. Für eine ungerade Zahl von einsen wird eine „1“ ausgegeben, sonst eine „0“.

x_2**x2**	x_1**x1**	x_0**x0**	y**y**
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1



Ein Schaltnetz ist eine schaltungstechnische Realisierung einer Schaltfunktion $f: \{0,1\}^n \rightarrow \{0,1\}^m$f : {0, 1}n → {0, 1}m mit $n, m \geq 1$n, m ≥ 1. ff ist zerlegbar in mm Boolesche Funktionen mit den gleichen n Eingangsvariablen: $f_1(x_1, x_2, ..., x_n), f_2(x_1, x_2, ..., x_n), ..., f_m(x_1, x_2, ..., x_n)$f1(x1, x2, ..., xn), f2(x1, x2, ..., xn), ..., fm(x1, x2, ..., xn)

Jedes Schaltnetz ist als gerichteter, azyklischer Graph darstellbar:

- Gatter, Ein- und Ausgänge sind die Knoten
- Verbindungsleitungen entsprechen den gerichteten Kanten
- Zyklen (Rückkopplungen) sind nicht zulässig!

Aus der Darstellung als kanonische Normalform resultiert, dass jede Schaltfunktion durch ein zweistufiges Schaltnetz realisierbar ist, wenn

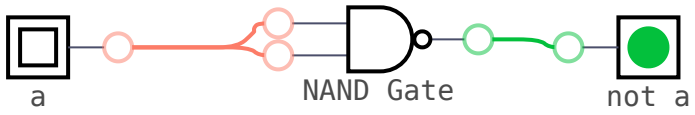
- alle Eingangssignale sowohl einfach als auch negiert vorliegen
- Gatter mit einer ausreichenden Größe (d.h. Anzahl an Eingängen)

zur Verfügung stehen

> **Merke:**
>
> - Jedes Schaltnetz kann nur mit UND-, ODER- und NICHT-Gattern aufgebaut werden.
> - Jedes Schaltnetz kann nur mit NAND Gattern aufgebaut werden.
> - Jedes Schaltnetz kann nur mit NOR Gattern aufgebaut werden.

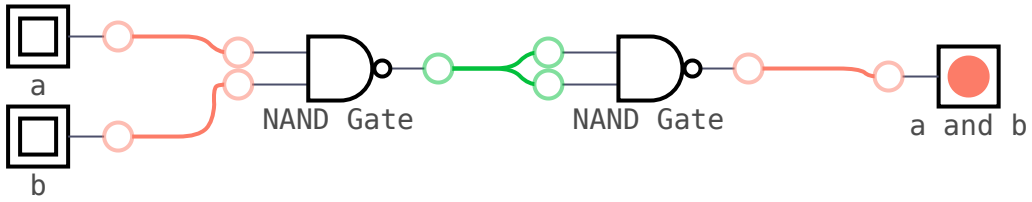
NOT

$\bar{a} = a \barwedge a$a = a · a



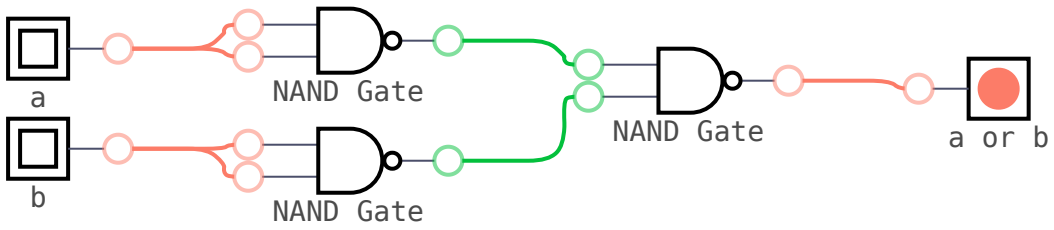
AND

$a \cdot b = a \barwedge a$a · b = a · a



OR

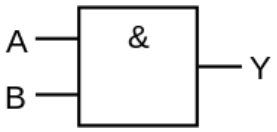
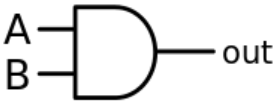
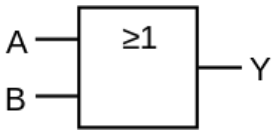
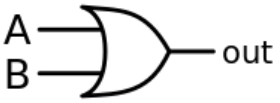
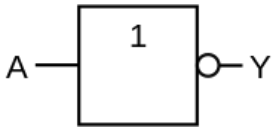
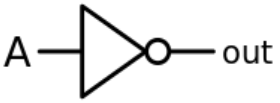
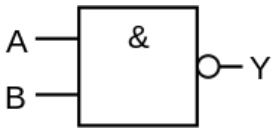
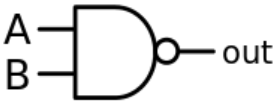
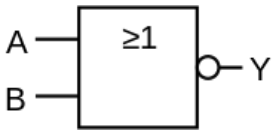
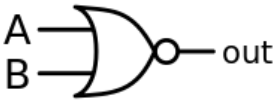
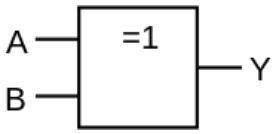
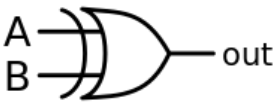
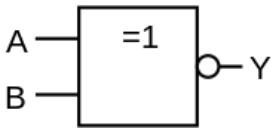
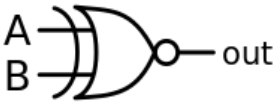
$a + b = \bar{a} \barwedge \bar{b}$a + b = a · b



Logikgatter als Grundlage der Schaltnetze

Ein Logikgatter, auch nur Gatter ist die technische Realisierung einer booleschen Funktion, die binäre Eingangssignale zu einem binären Ausgangssignal verarbeitet. Die Eingangssignale werden durch Implementierung logischer Operatoren, wie der Konjunktion (Und-Gatter), der Disjunktion (Oder-Gatter), der Kontravalenz (Exklusiv-Oder-Gatter) oder der Negation (Nicht-Gatter) zu einem einzigen logischen Ergebnis umgewandelt und auf das Ausgangssignal abgebildet.

Gatterfunktionen können zudem einen negierten Ausgang abbilden: NAND-Gatter (Nicht-Und), NOR-Gatter (Nicht-Oder), XNOR-Gatter (Nicht-Exklusiv-Oder).

Funktion	IEC 60617-12 : 1997	ANSI/IEEE Std 91/91a-1991
$UN \quad D$ $Y \quad = A \cdot B$ UNY Y $Y \quad = A \wedge B$ D = A · B = A ∧ B		
OR $Y \quad = A + B$ ORY Y $Y \quad = A \vee B$ = A + B = A ∨ B		
$NO \quad T$ $Y \quad = \bar{A}$ NOY Y T = A = A $Y \quad = A$		
$NA \quad ND$ $Y \quad = A \bar{\cdot} B$ NAY Y $Y \quad = A \bar{\wedge} B = A \bar{\wedge} B$ ND = A · B = A∧B = A ∧ B		
$NO \quad R$ $Y \quad = A \bar{+} B$ NOY Y $Y \quad = A \bar{\vee} B = A \bar{\vee} B$ R = A + B = A∨B = A ∨ B		
$XO \quad R$ $Y \quad = A \oplus B$ XOY Y $Y \quad = A \underline{\vee} B$ R = A ⊕ B = A∨B		
$XN \quad OR$ $Y \quad = A \odot B$ XNY Y $Y \quad = A \underline{\bar{\vee}} B = A \underline{\bar{\vee}} B$ OR = A ⊙ B = A∨B = A∨B		

Im englischen Sprachraum waren und sind die amerikanischen Symbole (rechte Spalte) üblich. Die IEC-Symbole sind international auf beschränkte Akzeptanz gestoßen und werden in der amerikanischen Literatur (fast) durchgängig ignoriert.

Zweistufige Schaltungssynthese

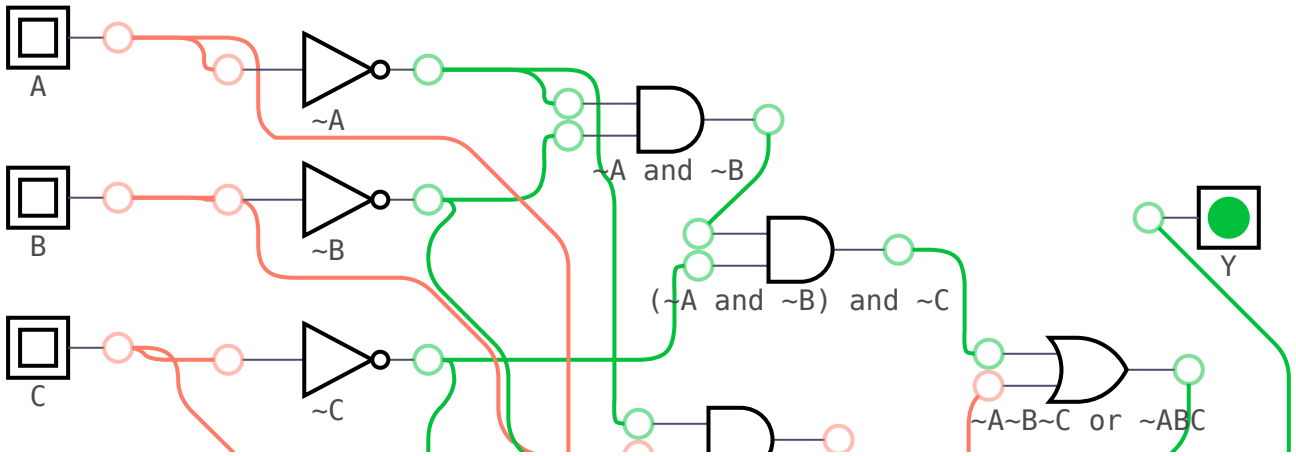
Dazu werden für eine KDNF je Minterm ein UND-Gatter und ein ODER-Gatter zur Disjunktion aller Minterme benötigt. Für die KKNF sind es entsprechend Maxterm ein ODER-Gatter ind ein UND-Gatter zur Konjunktion aller Maxterme.

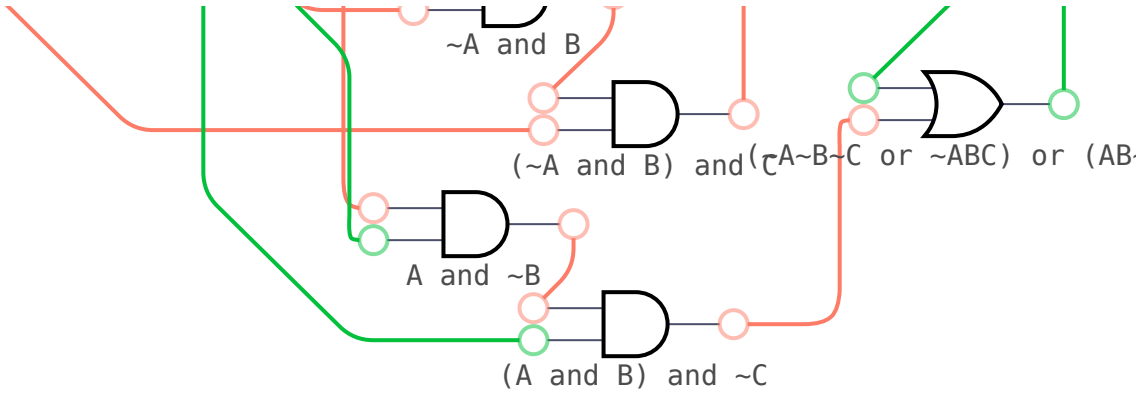
Die Anzahl der benötigten Gatter zur Realisierung der KDNF (bzw. KKNF) einer n-stelligen Schaltfunktion:

- je Boolescher Funktion max. 2^n2n UND-Gatter (bzw. ODER-Gatter) mit jeweils max. nn Eingängen
- je Boolescher Funktion ein ODER-Gatter (bzw. UND-Gatter) mit max. 2^n2n Eingängen

Beispiel:

A	B	C	Y
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	0



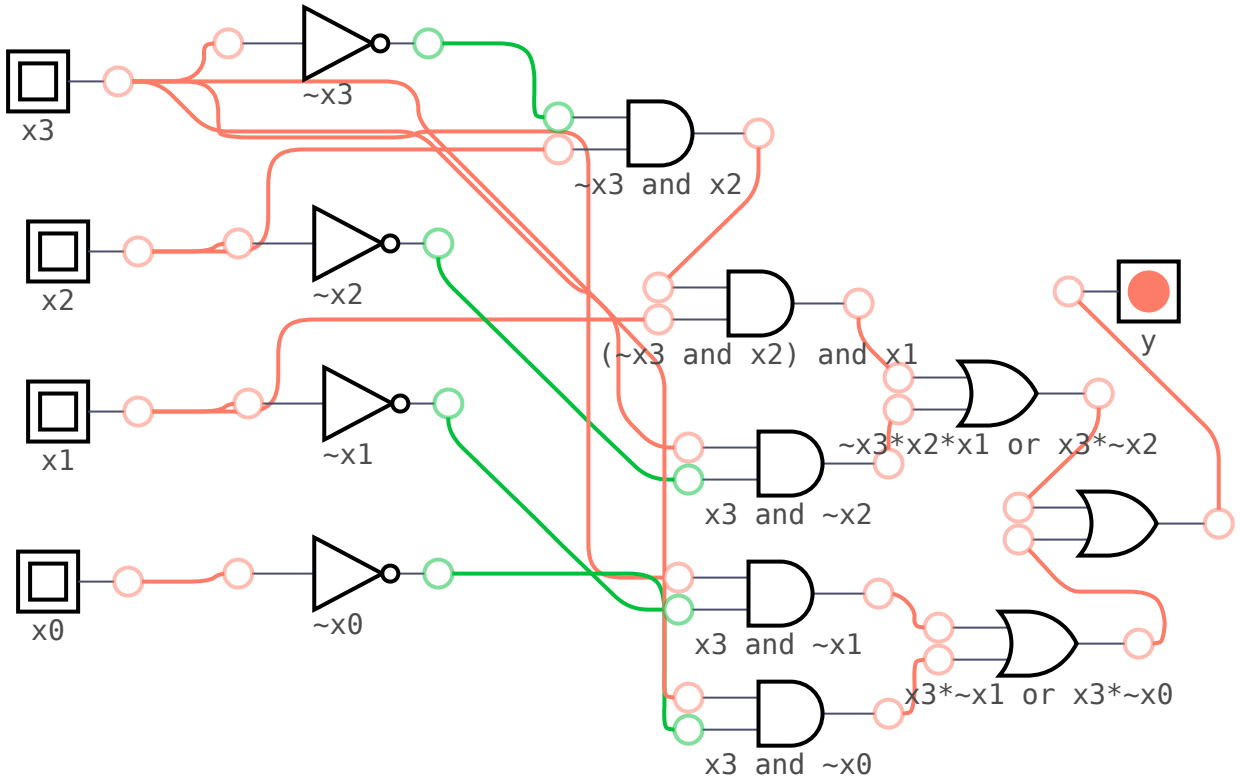


Zurück zum Beispiel - Schritt 3 Umsetzung

Für unser Kaffeemaschinenbeispiel kann eine Lösung also zum Beispiel folgendermaßen aussehen. Wir hatten ja folgende Funktion als minimale Darstellung identifiziert:

$$y = \bar{x}_3 x_2 x_1 + x_3 \bar{x}_2 + x_3 \bar{x}_1 + x_3 \bar{x}_0$$

y = x3x2x1 + x3 x2 + x3x1 + x3x0



Für die Zustände 6 bis 14 sollte ein Fehler ausgeben werden

x_3 **x 3**	x_2 **x2**	x_1 **x 1**	x_0 **x 0**	y **y**
0	0	0	0	0
…	…	…	…	0
0	1	0	1	0
0	1	1	0	1
0	1	1	1	1
1	0	0	0	1
1	0	0	1	1
1	0	1	0	1
1	0	1	1	1
1	1	0	0	1
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0

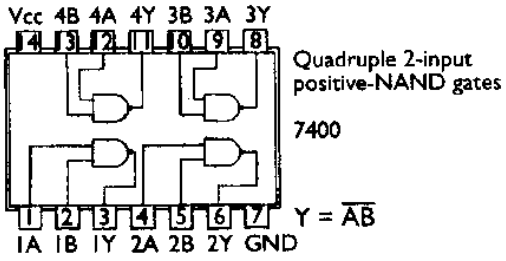
Worin unterscheidet sich unsere minimale Form von der technischen Umsetzung?

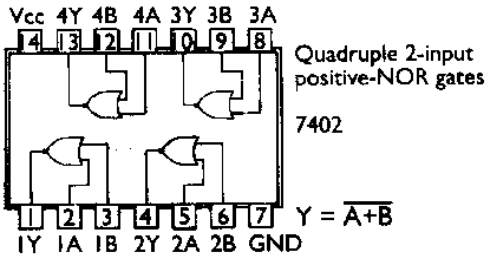
Spielen Sie ein wenig mit der Simulation, um sich von der Wirksamkeit unserer Lösung zu überzeugen.

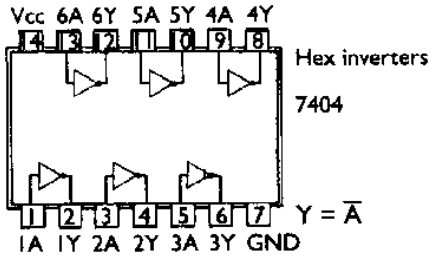
Umsetzung

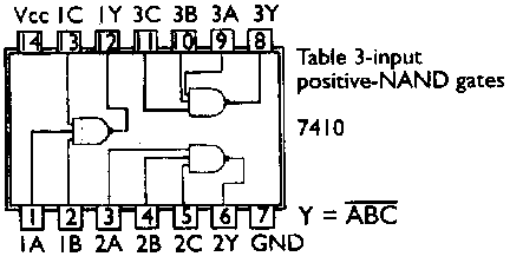
Prinzipiell lassen sich alle logischen Verknüpfungen als Gatter realisieren. Die 74xx Reihe umfasst dabei die Basisbausteine und darüber hinaus integrierte Schaltnetze, die höhere Funktionen bereitstellen (Addierwerk, Multiplexer, Decoder)

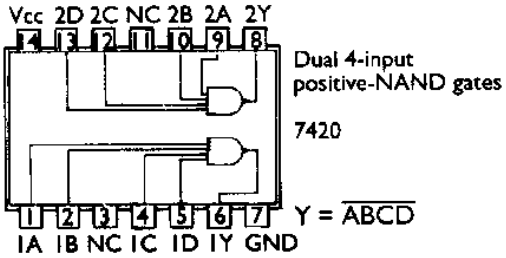
https://de.wikipedia.org/wiki/Liste_von_integrierten_Schaltkreisen_der_74xx-Familie

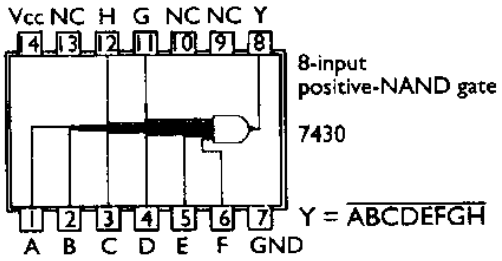




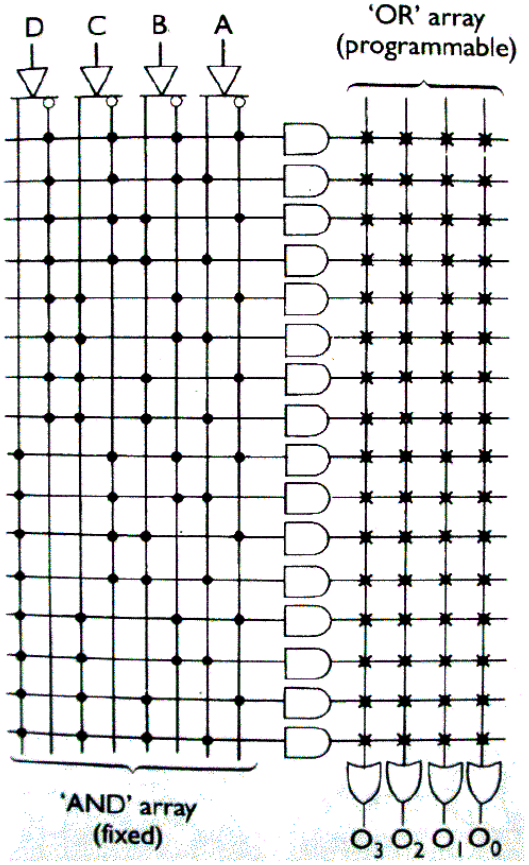
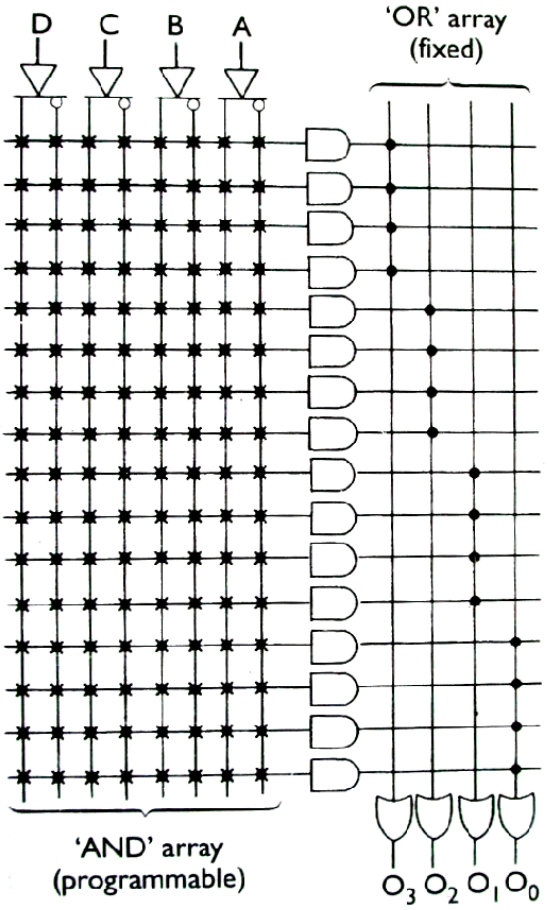






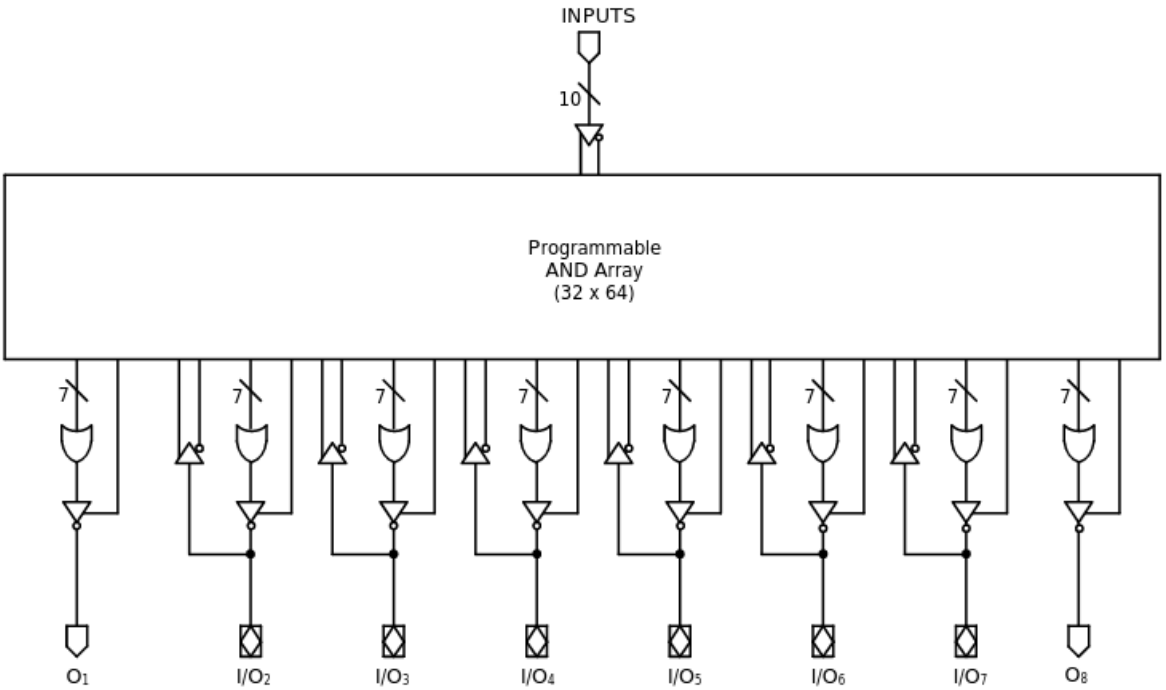


Aufwändigere Schaltnetze greifen auf ICs zurück, die zwei Ebenen AND-Array und OR-Array kombinieren.

PROM - „Programmable Read-Only Memory“	**PAL - „Programmable Array Logic“**
16 Worten zu 4 Bit	4 Ein- und Ausgängen und 4 Produkttermen je Ausgang
	
Adressdekoder entspricht einer festen UND-Matrix, Koppelelemente bilden eine programmierbare ODER-Matrix	programmierbare UND-Matrix, Produktterme werden mit fester ODER-Matrix verknüpft

PROM realisiert unmittelbar die Wahrheitstabelle in Hardware! Ein PROM mit 2^m2m nn-Bit Worten kann jede beliebige Schaltfunktion $f: \{0,1\}^n \to \{0,1\}^m$f : {0, 1}n → {0, 1}m ohne Minimierung implementieren.

PAL / GAL („Programmable / Generic Array Logic“ kann jede (ggf. minimierte) DNF realisieren, wenn Zahl der Produktterme je ODER ausreicht.



PAL16R8 Family, Advanced Micro Devices [1]

PAL wird eingesetzt:

- wenn viele Variablen und relative wenige Terme vorkommen.
- viele Eingangsbelegungen werden auf dieselbe Ausgangsbelegung abgebildet.

PROM wird eingesetzt:

- wenn jede Eingangsbelegung auf eine individuelle Ausgangsbelegung abgebildet werden muss.

[1] Datenblatt PAL16R8 Family, Advanced Micro Devices, link, 1996

Herausforderungen

In der Elektronik bezeichnet man mit Glitch eine kurzzeitige Falschaussage in logischen Schaltungen und temporäre Verfälschung einer booleschen Funktion. Diese tritt auf, weil die Signallaufzeiten in den einzelnen Gattern niemals vollkommen gleich sind.

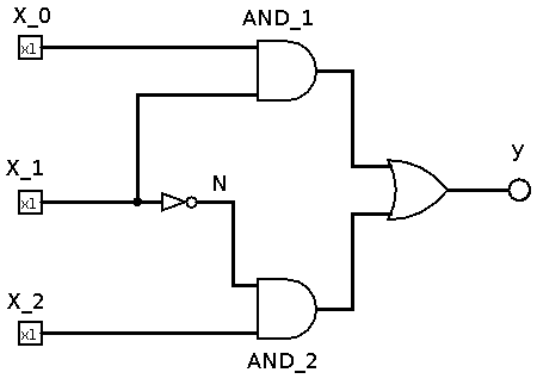
Ausgangspunkt

Zeitverhalten realer Logikbauteile

https://www.ti.com/lit/ds/symlink/sn74lvc2g08-ep.pdf

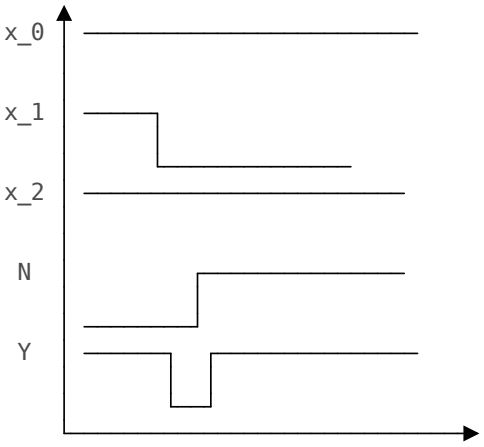
Konsequenz

Schaltwerk für die Umsetzung der Schaltfunktion $y = x_0x_1 + \bar{x}_1x_2$y = x0x1 + x1x2



Die Schaltung befindet sich zunächst in der Konfiguration $x_0 = x_1 = x_2 = 1$x0 = x1 = x2 = 1. Entsprechend liegt für yy ebenfalls eine $1$1 vor.

Nun wechselt x_1x1 auf „0“. Der Inverter benötigt allerdings eine gewisse Zeit, um den Ausgang von „1“ in eine „0“ umzusetzen. Für kurze Zeit ist sowohl $x_1 = 0$x1 = 0, als auch der Wert hinter dem Inverter „0“. Entsprechend schaltet „AND_2“ nicht durch, sondern gibt eine „0“ weiter an den Oder-Baustein.



.

Analyse im Karnaugh-Veitch-Diagramm

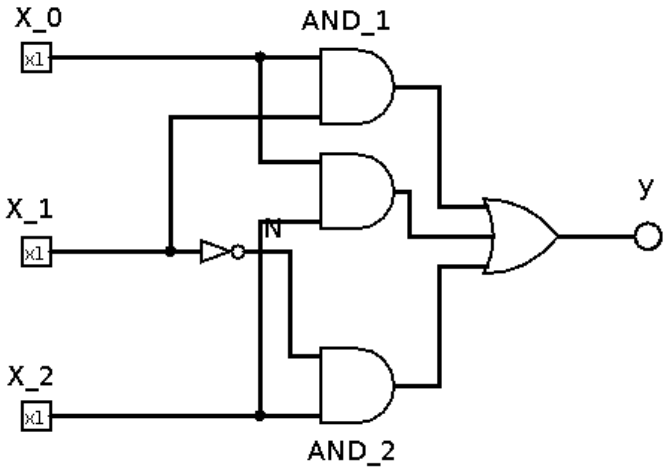
Das potentielle Auftreten eines Glitches wird im Karnaugh-Veitch Diagramm anhand nebeneinander liegender, nicht durch Überlappungen verbundener Schleifen deutlich.

	$\bar{x}_1\bar{x}_0$**x1 x0**	$\bar{x}_1x_0$**x1x0**	x_1x_0**x1x0**	$x_1\bar{x}_0$**x1x0**
$\bar{x}_2$x2	0	0	1	0
x_2x2	1	1	1	0

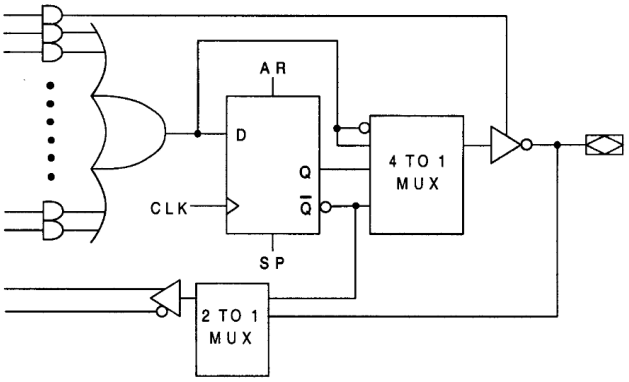
Lösungsansätze

1. Integration Glitch-Freier Pfade in dem wir die Überlappung im Karnaugh-Veitch Diagramm über den Minimierungsgedanken stellen.

$y = x_0x_1 + \bar{x}_1x_2 + x_0x_2$y = x0x1 + x1x2 + x0x2



2. Synchronisation gültiger Zustände



Was Sie heute mitnehmen

Diese Punkte fassen die Vorlesung zusammen. Die Fragen darunter gehören zu den Lernzielen vom Anfang: Wenn Sie sie beantworten und das Verfahren von Quine-McCluskey an einem eigenen Beispiel durchrechnen können, sind Sie gut auf die Standardschaltnetze in der nächsten Vorlesung vorbereitet.

- **Quine-McCluskey** minimiert eine KDNF algorithmisch: Minterme nach der Zahl ihrer Einsen sortieren, Terme, die sich in genau einer Variablen unterscheiden, stufenweise zusammenfassen – übrig bleiben die **Primimplikanten**.
- **Die Primimplikantentafel** zeigt, welcher Primimplikant welche Minterme abdeckt. **Essentielle Primimplikanten** decken einen Minterm als einzige ab und gehören immer zur Lösung; den Rest reduzieren Spalten- und Zeilendominanz.
- **Ein Schaltnetz** setzt eine Schaltfunktion als Schaltung ohne Rückkopplung um. Jede Schaltfunktion lässt sich zweistufig realisieren – und jedes Schaltnetz allein aus NAND- oder allein aus NOR-Gattern aufbauen.
- **PROM und PAL:** Ein PROM bildet die Wahrheitstafel unmittelbar ab und braucht keine Minimierung; ein PAL realisiert eine (minimierte) DNF über eine programmierbare UND-Matrix.
- **Glitches** entstehen, weil Gatter unterschiedliche **Signallaufzeiten** haben. Im Karnaugh-Veitch-Diagramm erkennt man sie an benachbarten Schleifen ohne Überlappung; ein zusätzlicher, überlappender Term oder die Synchronisation gültiger Zustände beseitigt sie.

Prüfen Sie sich:

- Erläutern Sie das Verfahren von Quine-McCluskey. Was ist ein Primimplikant, und wann ist er essentiell?
- Grenzen Sie die Begriffe Schaltnetz und Schaltfunktion voneinander ab.
- Welche Gatterkombinationen sind geeignet, um beliebige Schaltfunktionen damit umzusetzen? Bilden Sie ein ODER nur aus NAND-Gattern.
- Wann setzen Sie ein PROM, wann ein PAL ein?
- Was ist ein Glitch, und welche Rolle spielt die Laufzeitverzögerung der Gatter dabei?
- Wie können Sie Glitches in Schaltnetzen vermeiden?

Ihre Begriffe für die Kurslandkarte

Die Kurslandkarte wächst mit dem Semester, und Sie bestimmen mit, was hineinkommt. Ihre Antworten sammeln wir im Hörsaal gemeinsam über das LiaScript-Klassenzimmer; nach der Vorlesung werden die Begriffe dieser Woche in der Karte freigeschaltet.

Welche Begriffe aus der heutigen Vorlesung gehören für Sie in die Kurslandkarte?

Texteingabe ...

Wo begegnet Ihnen das, was heute dran war – in Ihrem Studiengang oder in Geräten Ihres Alltags?

Texteingabe ...